

EEE3096S: Practical 1A Demo
Phila Haanyama (HNYPHI001) and Lulama Lingela (LNGLUL002)

```
/**
*****
* @file      stm32f0xx_ll_utils.c
* @author    MCD Application Team
* @brief     UTILS LL module driver.
*****
* @attention
*
* Copyright (c) 2016 STMicroelectronics.
* All rights reserved.
*
* This software is licensed under terms that can be found in the LICENSE file
* in the root directory of this software component.
* If no LICENSE file comes with this software, it is provided AS-IS.
*
*****
*/

/* Includes -----*/
#include "stm32f0xx_ll_rcc.h"
#include "stm32f0xx_ll_utils.h"
#include "stm32f0xx_ll_system.h"
#ifdef USE_FULL_ASSERT
#include "stm32_assert.h"
#else
#define assert_param(expr) ((void)0U)
#endif

/** @addtogroup STM32F0xx_LL_Driver
* @{
*/

/** @addtogroup UTILS_LL
* @{
*/

/* Private types -----*/
/* Private variables -----*/
/* Private constants -----*/
/** @addtogroup UTILS_LL_Private_Constants
* @{
*/

/* Defines used for PLL range */
#define UTILS_PLL_OUTPUT_MIN      16000000U          /*!< Frequency min for PLL output, in Hz */
#define UTILS_PLL_OUTPUT_MAX      48000000U          /*!< Frequency max for PLL output, in Hz */

/* Defines used for HSE range */
#define UTILS_HSE_FREQUENCY_MIN    4000000U          /*!< Frequency min for HSE frequency, in Hz */
#define UTILS_HSE_FREQUENCY_MAX    32000000U          /*!< Frequency max for HSE frequency, in Hz */

/* Defines used for FLASH latency according to SYSCLOCK Frequency */
#define UTILS_LATENCY1_FREQ        24000000U          /*!< SYSCLOCK frequency to set FLASH latency 1 */
/**
* @}
*/

/* Private macros -----*/
/** @addtogroup UTILS_LL_Private_Macros
*/>
```

```

* @{
*/
#define IS_LL_UTILS_SYSCLOCK_DIV(__VALUE__) (((__VALUE__) == LL_RCC_SYSCLOCK_DIV_1) \
\
|| ((__VALUE__) == LL_RCC_SYSCLOCK_DIV_2) \
\
|| ((__VALUE__) == LL_RCC_SYSCLOCK_DIV_4) \
\
|| ((__VALUE__) == LL_RCC_SYSCLOCK_DIV_8) \
\
|| ((__VALUE__) == LL_RCC_SYSCLOCK_DIV_16) \
\
|| ((__VALUE__) == LL_RCC_SYSCLOCK_DIV_64) \
\
|| ((__VALUE__) == \
LL_RCC_SYSCLOCK_DIV_128) \
\
|| ((__VALUE__) == \
LL_RCC_SYSCLOCK_DIV_256) \
\
|| ((__VALUE__) == \
LL_RCC_SYSCLOCK_DIV_512))

#define IS_LL_UTILS_APB1_DIV(__VALUE__) (((__VALUE__) == LL_RCC_APB1_DIV_1) \
|| ((__VALUE__) == LL_RCC_APB1_DIV_2) \
|| ((__VALUE__) == LL_RCC_APB1_DIV_4) \
|| ((__VALUE__) == LL_RCC_APB1_DIV_8) \
|| ((__VALUE__) == LL_RCC_APB1_DIV_16))

#define IS_LL_UTILS_PLLMUL_VALUE(__VALUE__) (((__VALUE__) == LL_RCC_PLL_MUL_2) \
|| ((__VALUE__) == LL_RCC_PLL_MUL_3) \
|| ((__VALUE__) == LL_RCC_PLL_MUL_4) \
|| ((__VALUE__) == LL_RCC_PLL_MUL_5) \
|| ((__VALUE__) == LL_RCC_PLL_MUL_6) \
|| ((__VALUE__) == LL_RCC_PLL_MUL_7) \
|| ((__VALUE__) == LL_RCC_PLL_MUL_8) \
|| ((__VALUE__) == LL_RCC_PLL_MUL_9) \
|| ((__VALUE__) == LL_RCC_PLL_MUL_10) \
\
|| ((__VALUE__) == LL_RCC_PLL_MUL_11) \
\
|| ((__VALUE__) == LL_RCC_PLL_MUL_12) \
\
|| ((__VALUE__) == LL_RCC_PLL_MUL_13) \
\
|| ((__VALUE__) == LL_RCC_PLL_MUL_14) \
\
|| ((__VALUE__) == LL_RCC_PLL_MUL_15) \
\
|| ((__VALUE__) == LL_RCC_PLL_MUL_16))

#define IS_LL_UTILS_PREDIV_VALUE(__VALUE__) (((__VALUE__) == \
LL_RCC_PREDIV_DIV_1) || ((__VALUE__) == LL_RCC_PREDIV_DIV_2) || \
((__VALUE__) == \
LL_RCC_PREDIV_DIV_3) || ((__VALUE__) == LL_RCC_PREDIV_DIV_4) || \
((__VALUE__) == \
LL_RCC_PREDIV_DIV_5) || ((__VALUE__) == LL_RCC_PREDIV_DIV_6) || \
((__VALUE__) == \
LL_RCC_PREDIV_DIV_7) || ((__VALUE__) == LL_RCC_PREDIV_DIV_8) || \
((__VALUE__) == \
LL_RCC_PREDIV_DIV_9) || ((__VALUE__) == LL_RCC_PREDIV_DIV_10) || \
((__VALUE__) == \
LL_RCC_PREDIV_DIV_11) || ((__VALUE__) == LL_RCC_PREDIV_DIV_12) || \
((__VALUE__) == \
LL_RCC_PREDIV_DIV_13) || ((__VALUE__) == LL_RCC_PREDIV_DIV_14) || \
((__VALUE__) == \
LL_RCC_PREDIV_DIV_15) || ((__VALUE__) == LL_RCC_PREDIV_DIV_16))

```

```

#define IS_LL_UTILS_PLL_FREQUENCY(__VALUE__) ((UTILS_PLL_OUTPUT_MIN <=
(__VALUE__)) && ((__VALUE__) <= UTILS_PLL_OUTPUT_MAX))

#define IS_LL_UTILS_HSE_BYPASS(__STATE__) (((__STATE__) ==
LL_UTILS_HSEBYPASS_ON) \
                                         || ((__STATE__) ==
LL_UTILS_HSEBYPASS_OFF))

#define IS_LL_UTILS_HSE_FREQUENCY(__FREQUENCY__) (((__FREQUENCY__) >=
UTILS_HSE_FREQUENCY_MIN) && ((__FREQUENCY__) <= UTILS_HSE_FREQUENCY_MAX))
/**
 * @}
 */
/* Private function prototypes -----*/
/** @defgroup UTILS_LL_Private_Functions UTILS Private functions
 * @{
 */
static uint32_t      UTILS_GetPLLOutputFrequency(uint32_t PLL_InputFrequency,
                                                  LL_UTILS_PLLInitTypeDef
*UTILS_PLLInitStruct);
static ErrorStatus UTILS_EnablePLLAndSwitchSystem(uint32_t SYSCLOCK_Frequency,
LL_UTILS_ClkInitTypeDef *UTILS_ClkInitStruct);
static ErrorStatus UTILS_PLL_IsBusy(void);
/**
 * @}
 */

/* Exported functions -----*/
/** @defgroup UTILS_LL_Exported_Functions
 * @{
 */

/** @defgroup UTILS_LL_EF_DELAY
 * @{
 */

/**
 * @brief This function configures the Cortex-M SysTick source to have 1ms
time base.
 * @note When a RTOS is used, it is recommended to avoid changing the Systick
configuration by calling this function, for a delay use rather
osDelay RTOS service.
 * @param HCLKFrequency HCLK frequency in Hz
 * @note HCLK frequency can be calculated thanks to RCC helper macro or
function @ref LL_RCC_GetSystemClocksFreq
 * @retval None
 */
void LL_Init1msTick(uint32_t HCLKFrequency)
{
    /* Use frequency provided in argument */
    LL_InitTick(HCLKFrequency, 1000U);
}

/**
 * @brief This function provides accurate delay (in milliseconds) based
on SysTick counter flag
 * @note When a RTOS is used, it is recommended to avoid using blocking delay
and use rather osDelay service.
 * @note To respect 1ms timebase, user should call @ref LL_Init1msTick
function which
will configure Systick to 1ms
 * @param Delay specifies the delay time length, in milliseconds.
 * @retval None
 */
void LL_mDelay(uint32_t Delay)

```

```

{
    __IO uint32_t tmp = SysTick->CTRL; /* Clear the COUNTFLAG first */
    /* Add this code to indicate that local variable is not used */
    ((void)tmp);

    /* Add a period to guaranty minimum wait */
    if (Delay < LL_MAX_DELAY)
    {
        Delay++;
    }

    while (Delay)
    {
        if ((SysTick->CTRL & SysTick_CTRL_COUNTFLAG_Msk) != 0U)
        {
            Delay--;
        }
    }
}

/**
 * @}
 */

/** @addtogroup UTILS_EF_SYSTEM
 * @brief System Configuration functions
 *
 * @verbatim
=====
        ##### System Configuration functions #####
=====
    [...]
        System, AHB and APB buses clocks configuration

        (+) The maximum frequency of the SYSCLK, HCLK, PCLK1 and PCLK2 is
        48000000 Hz.
    @endverbatim
    @internal
        Depending on the SYSCLK frequency, the flash latency should be
        adapted accordingly:
        (++) +-----+
        (++) | Latency          | SYSCLK clock frequency (MHz) |
        (++) +-----+-----+
        (++) | 0WS(1CPU cycle) |      0 < SYSCLK <= 24      |
        (++) +-----+-----+
        (++) | 1WS(2CPU cycle) |     24 < SYSCLK <= 48     |
        (++) +-----+-----+
    @endinternal
    * @{
    */

/**
 * @brief This function sets directly SystemCoreClock CMSIS variable.
 * @note Variable can be calculated also through SystemCoreClockUpdate
function.
 * @param HCLKFrequency HCLK frequency in Hz (can be calculated thanks to RCC
helper macro)
 * @retval None
 */
void LL_SetSystemCoreClock(uint32_t HCLKFrequency)
{
    /* HCLK clock frequency */
    SystemCoreClock = HCLKFrequency;
}

/**

```

```

    * @brief Update number of Flash wait states in line with new frequency and
current voltage range.
    * @param Frequency SYSCLK frequency
    * @retval An ErrorStatus enumeration value:
    *         - SUCCESS: Latency has been modified
    *         - ERROR: Latency cannot be modified
    */
#ifdef FLASH_ACR_LATENCY
ErrorStatus LL_SetFlashLatency(uint32_t Frequency)
{
    uint32_t timeout;
    uint32_t getlatency;
    uint32_t latency;
    ErrorStatus status = SUCCESS;

    /* Frequency cannot be equal to 0 */
    if (Frequency == 0U)
    {
        status = ERROR;
    }
    else
    {
        if (Frequency > UTILS_LATENCY1_FREQ)
        {
            /* 24 < SYSCLK <= 48 => 1WS (2 CPU cycles) */
            latency = LL_FLASH_LATENCY_1;
        }
        else
        {
            /* else SYSCLK < 24MHz default LL_FLASH_LATENCY_0 0WS */
            latency = LL_FLASH_LATENCY_0;
        }
        LL_FLASH_SetLatency(latency);

        /* Check that the new number of wait states is taken into account to access
the Flash memory by reading the FLASH_ACR register */
        timeout = 2;
        do
        {
            /* Wait for Flash latency to be updated */
            getlatency = LL_FLASH_GetLatency();
            timeout--;
        } while ((getlatency != latency) && (timeout > 0));

        if (getlatency != latency)
        {
            status = ERROR;
        }
        else
        {
            /* No thing to do */
        }
    }

    return status;
}
#endif /* FLASH_ACR_LATENCY */

/**
    * @brief This function configures system clock with HSI as clock source of
the PLL
    * @note The application need to ensure that PLL is disabled.
    * @note Function is based on the following formula:
    *         - PLL output frequency = ((HSI frequency / PREDIV) * PLLMUL)

```

```

*           - PREDIV: Set to 2 for few devices
*           - PLLMUL: The application software must set correctly the PLL
multiplication factor to
*                   be in the range 16-48MHz
* @note     FLASH latency can be modified through this function.
* @param    UTILS_PLLInitStruct pointer to a @ref LL_UTILS_PLLInitTypeDef
structure that contains
*                   the configuration information for the PLL.
* @param    UTILS_ClkInitStruct pointer to a @ref LL_UTILS_ClkInitTypeDef
structure that contains
*                   the configuration information for the BUS
prescalers.
* @retval   An ErrorStatus enumeration value:
*           - SUCCESS: Max frequency configuration done
*           - ERROR: Max frequency configuration not done
*/
ErrorStatus LL_PLL_ConfigSystemClock_HSI(LL_UTILS_PLLInitTypeDef
*UTILS_PLLInitStruct,
                                           LL_UTILS_ClkInitTypeDef
*UTILS_ClkInitStruct)
{
    ErrorStatus status = SUCCESS;
    uint32_t pllfreq = 0U;

    /* Check if one of the PLL is enabled */
    if (UTILS_PLL_IsBusy() == SUCCESS)
    {
#if defined(RCC_PLLSRC_PREDIV1_SUPPORT)
        /* Check PREDIV value */
        assert_param(IS_LL_UTILS_PREDIV_VALUE(UTILS_PLLInitStruct->PLLDiv));
#else
        /* Force PREDIV value to 2 */
        UTILS_PLLInitStruct->Prediv = LL_RCC_PREDIV_DIV_2;
#endif /*RCC_PLLSRC_PREDIV1_SUPPORT*/
        /* Calculate the new PLL output frequency */
        pllfreq = UTILS_GetPLLOutputFrequency(HSI_VALUE, UTILS_PLLInitStruct);

        /* Enable HSI if not enabled */
        if (LL_RCC_HSI_IsReady() != 1U)
        {
            LL_RCC_HSI_Enable();
            while (LL_RCC_HSI_IsReady() != 1U)
            {
                /* Wait for HSI ready */
            }
        }

        /* Configure PLL */
#if defined(RCC_PLLSRC_PREDIV1_SUPPORT)
            LL_RCC_PLL_ConfigDomain_SYS(LL_RCC_PLLSOURCE_HSI, UTILS_PLLInitStruct-
>PLLMul, UTILS_PLLInitStruct->PLLDiv);
#else
            LL_RCC_PLL_ConfigDomain_SYS(LL_RCC_PLLSOURCE_HSI_DIV_2, UTILS_PLLInitStruct-
>PLLMul);
#endif /*RCC_PLLSRC_PREDIV1_SUPPORT*/

        /* Enable PLL and switch system clock to PLL */
        status = UTILS_EnablePLLAndSwitchSystem(pllfreq, UTILS_ClkInitStruct);
    }
    else
    {
        /* Current PLL configuration cannot be modified */
        status = ERROR;
    }

    return status;
}

```

```

}

#if defined(RCC_CFGR_SW_HSI48)
/**
 * @brief This function configures system clock with HSI48 as clock source of
the PLL
 * @note The application need to ensure that PLL is disabled.
 * @note Function is based on the following formula:
 * - PLL output frequency = ((HSI48 frequency / PREDIV) * PLLMUL)
 * - PLLMUL: The application software must set correctly the PLL
multiplication factor to
 * be in the range 16–48MHz
 * @param UTILS_PLLInitStruct pointer to a @ref LL_UTILS_PLLInitTypeDef
structure that contains
 * the configuration information for the PLL.
 * @param UTILS_ClkInitStruct pointer to a @ref LL_UTILS_ClkInitTypeDef
structure that contains
 * the configuration information for the BUS
prescalers.
 * @retval An ErrorStatus enumeration value:
 * - SUCCESS: Max frequency configuration done
 * - ERROR: Max frequency configuration not done
 */
ErrorStatus LL_PLL_ConfigSystemClock_HSI48(LL_UTILS_PLLInitTypeDef
*UTILS_PLLInitStruct,
LL_UTILS_ClkInitTypeDef
*UTILS_ClkInitStruct)
{
    ErrorStatus status = SUCCESS;
    uint32_t pllreq = 0U;

    /* Check if one of the PLL is enabled */
    if (UTILS_PLL_IsBusy() == SUCCESS)
    {
        /* Check PREDIV value */
        assert_param(IS_LL_UTILS_PREDIV_VALUE(UTILS_PLLInitStruct->PLLDiv));

        /* Calculate the new PLL output frequency */
        pllreq = UTILS_GetPLLOutputFrequency(HSI48_VALUE, UTILS_PLLInitStruct);

        /* Enable HSI48 if not enabled */
        if (LL_RCC_HSI48_IsReady() != 1U)
        {
            LL_RCC_HSI48_Enable();
            while (LL_RCC_HSI48_IsReady() != 1U)
            {
                /* Wait for HSI48 ready */
            }
        }

        /* Configure PLL */
        LL_RCC_PLL_ConfigDomain_SYS(LL_RCC_PLLSOURCE_HSI48, UTILS_PLLInitStruct->
        PLLMul, UTILS_PLLInitStruct->PLLDiv);

        /* Enable PLL and switch system clock to PLL */
        status = UTILS_EnablePLLAndSwitchSystem(pllreq, UTILS_ClkInitStruct);
    }
    else
    {
        /* Current PLL configuration cannot be modified */
        status = ERROR;
    }

    return status;
}

```

```

#endif /*RCC_CFGR_SW_HSI48*/
/**
 * @brief This function configures system clock with HSE as clock source of
the PLL
 * @note The application need to ensure that PLL is disabled.
 * @note Function is based on the following formula:
 * - PLL output frequency = ((HSE frequency / PREDIV) * PLLMUL)
 * - PLLMUL: The application software must set correctly the PLL
multiplication factor to
 * be in the range 16–48MHz
 * @note FLASH latency can be modified through this function.
 * @param HSEFrequency Value between Min_Data = 4000000 and Max_Data =
32000000
 * @param HSEBypass This parameter can be one of the following values:
 * @arg @ref LL_UTILS_HSEBYPASS_ON
 * @arg @ref LL_UTILS_HSEBYPASS_OFF
 * @param UTILS_PLLInitStruct pointer to a @ref LL_UTILS_PLLInitTypeDef
structure that contains
 * the configuration information for the PLL.
 * @param UTILS_ClkInitStruct pointer to a @ref LL_UTILS_ClkInitTypeDef
structure that contains
 * the configuration information for the BUS
prescalers.
 * @retval An ErrorStatus enumeration value:
 * - SUCCESS: Max frequency configuration done
 * - ERROR: Max frequency configuration not done
 */
ErrorStatus LL_PLL_ConfigSystemClock_HSE(uint32_t HSEFrequency, uint32_t
HSEBypass,
                                     LL_UTILS_PLLInitTypeDef
*UTILS_PLLInitStruct, LL_UTILS_ClkInitTypeDef *UTILS_ClkInitStruct)
{
    ErrorStatus status = SUCCESS;
    uint32_t pllfreq = 0U;

    /* Check the parameters */
    assert_param(IS_LL_UTILS_HSE_FREQUENCY(HSEFrequency));
    assert_param(IS_LL_UTILS_HSE_BYPASS(HSEBypass));

    /* Check if one of the PLL is enabled */
    if (UTILS_PLL_IsBusy() == SUCCESS)
    {
        /* Check PREDIV value */
        #if defined(RCC_PLLSRC_PREDIV1_SUPPORT)
            assert_param(IS_LL_UTILS_PREDIV_VALUE(UTILS_PLLInitStruct->PLLDiv));
        #else
            assert_param(IS_LL_UTILS_PREDIV_VALUE(UTILS_PLLInitStruct->Prediv));
        #endif /*RCC_PLLSRC_PREDIV1_SUPPORT*/

        /* Calculate the new PLL output frequency */
        pllfreq = UTILS_GetPLLOutputFrequency(HSEFrequency, UTILS_PLLInitStruct);

        /* Enable HSE if not enabled */
        if (LL_RCC_HSE_IsReady() != 1U)
        {
            /* Check if need to enable HSE bypass feature or not */
            if (HSEBypass == LL_UTILS_HSEBYPASS_ON)
            {
                LL_RCC_HSE_EnableBypass();
            }
            else
            {
                LL_RCC_HSE_DisableBypass();
            }
        }

        /* Enable HSE */

```



```

    LL_RCC_HSE_Enable();
    while (LL_RCC_HSE_IsReady() != 1U)
    {
        /* Wait for HSE ready */
    }
}

/* Configure PLL */
#if defined(RCC_PLLSRC_PREDIV1_SUPPORT)
    LL_RCC_PLL_ConfigDomain_SYS(LL_RCC_PLLSOURCE_HSE, UTILS_PLLInitStruct->PLLMul, UTILS_PLLInitStruct->PLLDiv);
#else
    LL_RCC_PLL_ConfigDomain_SYS((RCC_CFGR_PLLSRC_HSE_PREDIV |
    UTILS_PLLInitStruct->Prediv), UTILS_PLLInitStruct->PLLMul);
#endif /*RCC_PLLSRC_PREDIV1_SUPPORT*/

    /* Enable PLL and switch system clock to PLL */
    status = UTILS_EnablePLLAndSwitchSystem(pllfreq, UTILS_ClkInitStruct);
}
else
{
    /* Current PLL configuration cannot be modified */
    status = ERROR;
}

return status;
}

/**
 * @}
 */

/**
 * @}
 */

/** @addtogroup UTILS_LL_Private_Functions
 * @{
 */

/**
 * @brief Function to check that PLL can be modified
 * @param PLL_InputFrequency PLL input frequency (in Hz)
 * @param UTILS_PLLInitStruct pointer to a @ref LL_UTILS_PLLInitTypeDef
structure that contains
 * the configuration information for the PLL.
 * @retval PLL output frequency (in Hz)
 */
static uint32_t UTILS_GetPLLOutputFrequency(uint32_t PLL_InputFrequency,
LL_UTILS_PLLInitTypeDef *UTILS_PLLInitStruct)
{
    uint32_t pllfreq = 0U;

    /* Check the parameters */
    assert_param(IS_LL_UTILS_PLLMUL_VALUE(UTILS_PLLInitStruct->PLLMul));

    /* Check different PLL parameters according to RM */
    /* The application software must set correctly the PLL multiplication factor
to
    be in the range 16-48MHz */
    #if defined(RCC_PLLSRC_PREDIV1_SUPPORT)
        pllfreq = __LL_RCC_CALC_PLLCLK_FREQ(PLL_InputFrequency, UTILS_PLLInitStruct->PLLMul, UTILS_PLLInitStruct->PLLDiv);
    #else
        pllfreq = __LL_RCC_CALC_PLLCLK_FREQ(PLL_InputFrequency / (UTILS_PLLInitStruct->Prediv + 1U), UTILS_PLLInitStruct->PLLMul);
    #endif /*RCC_PLLSRC_PREDIV1_SUPPORT*/

```

```

    assert_param(IS_LL_UTILS_PLL_FREQUENCY(pllfreq));

    return pllfreq;
}

/**
 * @brief Function to check that PLL can be modified
 * @retval An ErrorStatus enumeration value:
 *         - SUCCESS: PLL modification can be done
 *         - ERROR: PLL is busy
 */
static ErrorStatus UTILS_PLL_IsBusy(void)
{
    ErrorStatus status = SUCCESS;

    /* Check if PLL is busy */
    if (LL_RCC_PLL_IsReady() != 0U)
    {
        /* PLL configuration cannot be modified */
        status = ERROR;
    }

    return status;
}

/**
 * @brief Function to enable PLL and switch system clock to PLL
 * @param SYSCLK_Frequency SYSCLK frequency
 * @param UTILS_ClkInitStruct pointer to a @ref LL_UTILS_ClkInitTypeDef
        structure that contains
        * the configuration information for the BUS
        prescalers.
 * @retval An ErrorStatus enumeration value:
 *         - SUCCESS: No problem to switch system to PLL
 *         - ERROR: Problem to switch system to PLL
 */
static ErrorStatus UTILS_EnablePLLAndSwitchSystem(uint32_t SYSCLK_Frequency,
LL_UTILS_ClkInitTypeDef *UTILS_ClkInitStruct)
{
    ErrorStatus status = SUCCESS;
    uint32_t sysclk_frequency_current = 0U;

    assert_param(IS_LL_UTILS_SYSCLK_DIV(UTILS_ClkInitStruct->AHBCLKDivider));
    assert_param(IS_LL_UTILS_APB1_DIV(UTILS_ClkInitStruct->APB1CLKDivider));

    /* Calculate current SYSCLK frequency */
    sysclk_frequency_current = (SystemCoreClock <<
AHBPrescTable[LL_RCC_GetAHBPrescaler() >> RCC_POSITION_HPRE]);

    /* Increasing the number of wait states because of higher CPU frequency */
    if (sysclk_frequency_current < SYSCLK_Frequency)
    {
        /* Set FLASH latency to highest latency */
        status = LL_SetFlashLatency(SYSCLK_Frequency);
    }

    /* Update system clock configuration */
    if (status == SUCCESS)
    {
        /* Enable PLL */
        LL_RCC_PLL_Enable();
        while (LL_RCC_PLL_IsReady() != 1U)
        {
            /* Wait for PLL ready */
        }
    }
}

```

```

/* Sysclk activation on the main PLL */
LL_RCC_SetAHBPrescaler(UTILS_ClkInitStruct->AHBCLKDivider);
LL_RCC_SetSysClkSource(LL_RCC_SYS_CLKSOURCE_PLL);
while (LL_RCC_GetSysClkSource() != LL_RCC_SYS_CLKSOURCE_STATUS_PLL)
{
    /* Wait for system clock switch to PLL */
}

/* Set APB1 & APB2 prescaler*/
LL_RCC_SetAPB1Prescaler(UTILS_ClkInitStruct->APB1CLKDivider);
}

/* Decreasing the number of wait states because of lower CPU frequency */
if (sysclk_frequency_current > SYSCLK_Frequency)
{
    /* Set FLASH latency to lowest latency */
    status = LL_SetFlashLatency(SYSCLK_Frequency);
}

/* Update SystemCoreClock variable */
if (status == SUCCESS)
{
    LL_SetSystemCoreClock(__LL_RCC_CALC_HCLK_FREQ(SYSCLK_Frequency,
UTILS_ClkInitStruct->AHBCLKDivider));
}

return status;
}

/**
 * @}
 */

/**
 * @}
 */

/**
 * @}
 */

```